

Verificação de uma classe de frações: geração aleatória guiada por feedback e teste baseado em propriedades

Eduardo Hoffmann, Lucas Mocelin e Fernando Kunst

Verificação e Validação – PUCRS, Escola Politécnica, 2026 — Trabalho T1

Repositório: github.com/theeduardohoffmann/fraction-lab

1 Introdução

O kata escolhido é *Fraction*, um objeto de valor imutável que representa números racionais com numerador e denominador long, sempre normalizados: denominador positivo, fração irredutível e zero como 0/1. A classe oferece criação (of, parse), aritmética (plus, minus, times, dividedBy), ordem e igualdade por valor, e sinaliza overflow com ArithmeticException. O código é compatível com Java 17 e usa Maven, JUnit 5, jqwik 1.9.3, Randoop 4.3.4 e JaCoCo; o repositório tem um workflow de integração contínua configurado no GitHub Actions. Os experimentos foram executados com JDK 21, no macOS.

O objetivo é aplicar duas técnicas ao mesmo problema, compará-las criticamente e justificar as escolhas: (1) geração aleatória guiada por feedback, com contratos como oráculo, e (2) teste baseado em propriedades. A classe é pequena, mas reúne invariantes fortes, uma álgebra bem conhecida (o corpo dos racionais) e um modo de falha numérico (overflow), o que permite comparar oráculos genéricos e propriedades do domínio.

2 Referencial

Pacheco et al. (2007) propõem a geração aleatória guiada por feedback, implementada no Randoop. Sequências de chamadas são montadas aos poucos e executadas, e o resultado de cada execução orienta a geração seguinte. O oráculo é um conjunto de contratos (a Figura 4 do artigo lista: o.equals(o) verdadeiro; equals, hashCode e toString sem exceção; nenhum NullPointerException quando nenhum argumento era nulo; nenhum AssertionError), de modo que ninguém precisa escrever o resultado esperado, e o usuário pode acrescentar contratos do domínio. Três filtros evitam entradas inúteis: o de igualdade descarta objetos iguais (por equals) a outros já criados, o de nulos impede reaproveitar null e o de exceções impede que sequências que lançam exceção sejam estendidas, pois a exceção costuma indicar violação de pré-condição. Para argumentos primitivos, o gerador sorteia valores de um conjunto fixo e pequeno (como -1, 0 e 1), que o usuário pode ampliar. Pontos fortes: automação, baixo custo de especificação e detecção de falhas em bibliotecas reais (14 bibliotecas, 780 mil linhas). Limitações: sem contratos adicionais, o oráculo diz pouco sobre a correção funcional; o filtro de exceções supõe que exceção significa entrada inválida; e o próprio artigo admite que o filtro de igualdade pode fazer a ferramenta perder erros, se objetos iguais se comportarem de modo diferente.

Fink e Bishop (1997) apresentam o teste baseado em propriedades como forma de assegurar que um programa está livre de falhas especificadas. O analista escreve a propriedade em TASPEC; muitas propriedades são genéricas (limites de vetor, condições de corrida, autenticação) e podem formar bibliotecas reutilizáveis, mas propriedades específicas do programa também podem ser usadas. O programa é fatiado para isolar o código relevante, um monitor de execução gerado da especificação faz o papel de oráculo e a métrica de cobertura de fluxo de dados iterative contexts indica quando o teste está completo. O estudo de caso é um defeito de autenticação no ftpd (cerca de 3000 linhas de C, grafo de fluxo de dados com 6148 nós e 31912 arestas). Pontos fortes: critério explícito de completude e oráculo derivado da especificação. Limitações: os números descrevem um único estudo de caso, sem avaliação experimental comparativa; os autores deixam em aberto como escolher as propriedades e como saber se elas cobrem as falhas possíveis; e a geração de dados para as lacunas de cobertura não é automática.

Comparação. Os dois trabalhos usam a especificação como oráculo, mas com ênfases diferentes, e um não substitui o outro. Fink e Bishop concentram o esforço na análise do programa: a propriedade guia o fatiamento e define um critério de completude, enquanto a escolha dos dados fica com o testador. Pacheco et al. concentram o esforço na geração automática de entradas. Nos dois casos, a especificação pode ser genérica ou específica: Fink e Bishop usam bibliotecas de propriedades genéricas e propriedades do programa, e o Randoop tem contratos padrão que podem ser complementados por contratos do domínio. Uma ressalva: o jqwik não implementa a técnica de Fink e Bishop (não há TASPEC, fatiamento nem iterative contexts). Ele gera entradas aleatórias a partir de geradores declarados e verifica propriedades algébricas. Do artigo aproveitamos a ideia central, a propriedade como especificação e oráculo, e a ligação com o jqwik é uma interpretação da equipe, não uma afirmação dos autores. O artigo de Pacheco et al. cita o QuickCheck, ferramenta para Haskell à qual o jqwik se assemelha.

3 Aplicação

Randoop. O artigo de 2007 descreve os contratos padrão da Figura 4; deles usamos C1 (o.equals(o) verdadeiro), C5 (equals, hashCode e toString sem exceção) e C6 (nenhum NullPointerException quando nenhum argumento é null). Na versão 4.3.4, C6 foi configurado explicitamente com --npe-on-non-null-input=ERROR; as demais exceções seguiram o tratamento padrão da versão. A versão 4.3.4 verifica ainda simetria e transitividade de equals, hashCode consistente (C2–C4) e contratos de compareTo, que o artigo não lista. Registramos quatro contratos do domínio como pós-condições em JSON: denominador positivo (C7), irreduzibilidade (C8), compareTo coerente com equals (C9) e ida e volta parse(toString()) (C10). Como o conjunto padrão de primitivos é pequeno, fornecemos um arquivo de literais (± 1000 , ± 7 , Long.MAX_VALUE e Long.MIN_VALUE, entre outros). As execuções principais usaram limite de 60 segundos e semente 42 (o padrão descrito no artigo é de 2 minutos). Para M1 e M4, também foram realizadas execuções com sementes 1, 2 e 3. Os testes gerados (JUnit 4) rodam pelo motor vintage.

jqwik. Formulamos 14 propriedades (P1–P14), verificadas por 18 métodos de teste configurados para até 1.000 tentativas por propriedade: normalização, igualdade por valor, ida e volta, comutatividade, associatividade, elementos neutros, inversos, distributividade, ordem total, divisão e exceções. As propriedades algébricas usam geradores pequenos ($|n| \leq 1000$, $1 \leq d \leq 1000$) para isolar a álgebra do overflow. O overflow é tratado em propriedades separadas (P1, P10d, P13), com a faixa completa de long e uma referência em BigInteger: o resultado retornado deve ser exato, ou a operação deve lançar ArithmeticException. As sementes efetivas do jqwik ficam registradas nos logs de execução.

Baseline. 15 testes de exemplo por partição de equivalência e valor limite (EX-01 a EX-15), para contraste. O arquivo tests.md liga cada teste a técnica, contrato ou propriedade e requisito (R1–R15).

4 Discussão

Resultados. Na execução de 02/10/2026 com o código correto, os 15 testes por exemplos e os 18 métodos de propriedades passaram, e o Randoop gerou 1.467 testes de regressão e nenhum teste revelador de erro: 1.500 testes executados, sem falhas ou erros. Como a geração do Randoop é limitada a 60 s, as quantidades geradas também dependem da carga da máquina; as execuções foram feitas em cópias isoladas do projeto, com até duas execuções independentes em paralelo. Para avaliar a capacidade de detecção, aplicamos cinco mutações, das quais quatro introduzem defeitos, e consideramos "detectada" a mutação em que ao menos um teste da técnica falha (Tabela 1).

Tabela 1 – Resultado das cinco mutações, das quais quatro introduzem defeitos (Randoop: 60 s, semente 42)

Mutação	Exemplos	jqwik	Randoop (testes reveladores de erro)
M1: sinal do denominador não normalizado	EX-01	P1, P2, P3, P11	18 falharam
M2: sem redução por mdc	EX-02, EX-07, EX-14, EX-15	P1, P2, P5, P9, P11	16 falharam
M3: compareTo por multiplicação em long	não detectou	P10d	4 falharam
M4: overflow ignorado em plus e times	EX-12	P13	não detectou
M5: hashCode só do numerador (preserva o contrato de igualdade/hashCode)	não detectou	não detectou	não detectou

Fonte: execuções da equipe (tests.md e relatório dos experimentos).

Contratos × propriedades. M4 mostra um limite dos contratos usados: nenhum deles verifica o resultado aritmético, e o Randoop não detectou o defeito nas sementes 1, 2, 3 e 42, enquanto P13 o detectou porque compara o resultado com uma referência exata. Esse resultado vale para esses contratos e sementes; contratos aritméticos adicionais poderiam mudá-lo. Em M3, a situação se inverte em parte: entre as propriedades, só P10d, que usa a faixa larga, detectou o defeito, enquanto o Randoop o detectou com 4 testes reveladores de erro. M1 foi detectado pelo Randoop nas sementes 1, 2, 3 e 42. Neste experimento, a faixa das entradas também influenciou a detecção, e não apenas o oráculo. M5 muda o hashCode, mas preserva o contrato de igualdade/hashCode: como a forma é normalizada, frações iguais mantêm o mesmo hash, e nenhuma técnica detectou a mudança.

Custo de especificação. No Randoop, a especificação do domínio coube em quatro entradas JSON e um arquivo de literais; nas propriedades, foram 18 métodos de teste, geradores com faixas escolhidas e um oráculo em BigInteger, o que exigiu modelar a aritmética e o risco de overflow. Não medimos o esforço de cada abordagem, então essa comparação é apenas qualitativa.

Exceções e regressão. Divisão por zero, recíproco de zero e overflow lançam exceção por especificação, não por entrada inválida. Conforme o artigo, o filtro de exceções impede que sequências que lançam exceção sejam estendidas, o que pode reduzir a exploração perto do overflow; o experimento, porém, não isola essa causa. Os testes de regressão do Randoop registram comportamentos observados, e passarem numa versão mutada não prova que ela esteja correta. P13 verifica a exatidão dos resultados retornados, mas aceita ArithmeticException sem demonstrar, em cada caso, que a exceção era necessária; P12, EX-04, EX-10 e EX-12 verificam exceções exigidas em casos específicos.

Dificuldades. (i) A primeira versão da pós-condição C10 lançava exceção ao avaliar parse sobre texto malformato, e o Randoop abortou; foi preciso uma guarda de formato. (ii) Com o conjunto padrão de primitivos, o gerador usou quase só of(1, 1) e não detectou M1 numa execução de 20 s, o que nos levou a fornecer literais. (iii) Foi necessário decidir o tratamento de Long.MIN_VALUE (o numerador nunca pode assumi-lo) para que negate e abs sejam seguros. (iv) Mesmo com a mesma semente, o Randoop produz quantidades diferentes de sequências, porque limita a geração por tempo.

Ameaças à validade. Interna: as mutações foram feitas à mão, são cinco (quatro introduzem defeitos), e "detectada" significa apenas "algum teste falhou". Externa: uma única classe pequena e numérica. De construção: as mutações representam alguns tipos de defeito, e

as propriedades foram escritas pela mesma equipe que escreveu a classe. De conclusão: poucas repetições (quatro sementes do Randoop para M1 e M4, uma para os demais) e nenhum tratamento estatístico.

5 Conclusão

As técnicas se complementaram neste experimento. O Randoop, com uma especificação curta (quatro contratos em JSON e um arquivo de literais), verificou contratos de objeto, contratos do domínio e sequências de chamadas, mas os contratos usados não verificam o resultado aritmético, e M4 não foi detectado nas sementes testadas. As propriedades verificaram a correção algébrica e a exatidão diante de overflow, ao custo de modelar o domínio e escolher bem os geradores. Aprendemos que (a) a escolha das entradas influenciou a detecção (literais no Randoop, faixa nos geradores do jqwik); (b) as mutações ajudaram a avaliar a capacidade de detecção dos oráculos neste experimento; e (c) uma alteração que preserva os contratos verificados, como M5, não é detectada por oráculos baseados neles. Esses resultados valem para as cinco mutações, das quais quatro introduzem defeitos, e para a classe estudada. A comparação mostra ênfases diferentes entre os artigos, e não a substituição de uma abordagem pela outra.

Referências

FINK, G.; BISHOP, M. Property-based testing: a new approach to testing for assurance. **ACM SIGSOFT Software Engineering Notes**, New York, v. 22, n. 4, p. 74-80, jul. 1997. DOI: 10.1145/263244.263267. Disponível em: <https://nob.cs.ucdavis.edu/bishop/papers/1997-sen>. Acesso em: 2 out. 2026.

PACHECO, C.; LAHIRI, S. K.; ERNST, M. D.; BALL, T. Feedback-directed random test generation. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING, 29., 2007, Minneapolis. **Proceedings** [...]. 2007. p. 75-84. Disponível em: <https://homes.cs.washington.edu/~mernst/pubs/feedback-testgen-icse2007.pdf>. Acesso em: 2 out. 2026.